PROFESSIONAL SKILLS - II

MINI PROJECT REPORT

SMART CLOUD FILE MANAGER

Serverless Cloud File Storage and Metadata Management System

Submitted by

ROLL NO.	NAME
07	Ishita Patel
09	Rashmi Pathak
13	Kaushal Pawar
22	Shriya Prasad

Under the Guidance of

Santanu Das

Thakur College of Engineering and Technology

Academic Year 2026-2027

TABLE OF CONTENTS

The report follows the prescribed Internet Programming mini-project format.

1. ABSTRACT . . . 3

2. INTRODUCTION . . . 4

2.1 Background and Motivation . . . 4

2.2 Problem Statement . . . 4

2.3 Objectives . . . 4

2.4 Scope of the Project . . . 4

3. PROJECT DESCRIPTION . . . 5

3.1 System Overview and Core Architecture . . . 5

3.2 Main Features and Functional Capabilities . . . 5

3.3 Technologies Used and Cloud Infrastructure Stack . . . 6

3.4 Serverless Cloud Design: API Gateway, Lambda, S3 and DynamoDB . . . 6

3.5 System Workflow and Presigned URL Lifecycle . . . 6

3.6 Security, Versioning and Cost Awareness . . . 6

4. RESULTS AND DISCUSSIONS (SCREENSHOTS OF YOUR PROJECT) . . . 7

4.1 Application Screenshots and Cloud Architecture Verification . . . 7

4.2 Functional Test Summary . . . 10

4.3 Discussion and Architectural Evaluation . . . 10

5. CONCLUSION AND FUTURE SCOPE . . . 11

5.1 Conclusion . . . 11

5.2 Future Scope . . . 11

5.3 Learning Outcomes . . . 11

6. REFERENCES . . . 12

7. ANNEXURE (CODE) . . . 13

1. ABSTRACT

Smart Cloud File Manager is a browser-based file management prototype that demonstrates how a web interface can use serverless cloud services to upload, list, and download files. The project uses React and Vite for the local frontend, Amazon API Gateway to receive HTTP requests, AWS Lambda with Python and boto3 for backend processing, Amazon S3 to store file objects, and Amazon DynamoDB to store searchable file metadata.

During development, the team created a private S3 bucket with versioning enabled, configured a DynamoDB metadata table, created a Lambda execution role, deployed the Python handler, and connected the handler through an HTTP API. A CORS configuration was added to S3 to permit browser uploads from the local development origin. The handler generates short-lived presigned URLs so the browser can transfer a file directly to S3 without sending the file contents through Lambda.

The prototype was functionally verified for a small text-file upload, display in the website file list, appearance in the S3 bucket, and download. The report documents the architecture, workflow, results, code structure, security considerations, and cost awareness. Version listing, restore, and delete behavior are included in the backend route design but require separate end-to-end verification before being reported as complete.

Keywords: Internet Programming, Cloud Computing, Amazon S3, AWS Lambda, API Gateway, Amazon DynamoDB, Presigned URLs, Serverless Architecture.

2. INTRODUCTION

2.1 Background and Motivation

Modern applications increasingly store user data in cloud services rather than on a single personal computer. A cloud file manager provides a simple interface for placing files in remote object storage and retrieving them later. Such a system also demonstrates core Internet Programming concepts: frontend-backend communication, request handling, data persistence, access control, and integration of managed services.

The project was selected as a practical way to connect a browser interface with a backend API and cloud resources. It separates the actual file contents from the descriptive information used to display each file, making the design easier to understand and maintain.

2.2 Problem Statement

A file-management interface that stores data only in the browser or on a single machine is limited to that environment and does not demonstrate managed cloud storage. A cloud-based prototype must securely store the file contents, record metadata, provide a reliable upload/download path, and allow the frontend to call backend operations without browser cross-origin failures.

2.3 Objectives

- Develop a browser interface for uploading, listing, and downloading files.
- Store file contents in an Amazon S3 bucket and preserve object versions through S3 Versioning.
- Use AWS Lambda to generate time-limited presigned URLs and process file-management requests.
- Store file metadata in Amazon DynamoDB so the website can present a file list.
- Connect frontend and backend through Amazon API Gateway using an HTTP API route.
- Keep the bucket private, limit backend permissions to the project resources, and monitor AWS usage and credits.

2.4 Scope of the Project

The project scope covers the local React/Vite website and its integration with API Gateway, Lambda, S3, DynamoDB, and IAM. The implemented prototype was tested for a small-file upload, metadata listing, and download. The website currently runs at localhost:5173 for demonstration and is not deployed as a public website. Authentication and per-user authorization are not implemented, so the application should not be used to store confidential files or exposed for public use without further safeguards.

3. PROJECT DESCRIPTION

3.1 System Overview and Core Architecture

The system follows a serverless, decoupled design. React and Vite provide the user interface. API Gateway exposes an HTTP endpoint and forwards request routes to a Lambda function. Lambda performs validation and coordination, generates presigned S3 URLs, and reads or writes metadata in DynamoDB. The browser transfers file bytes directly to S3 using the temporary URL. This separation prevents the whole file from having to pass through the Lambda runtime.

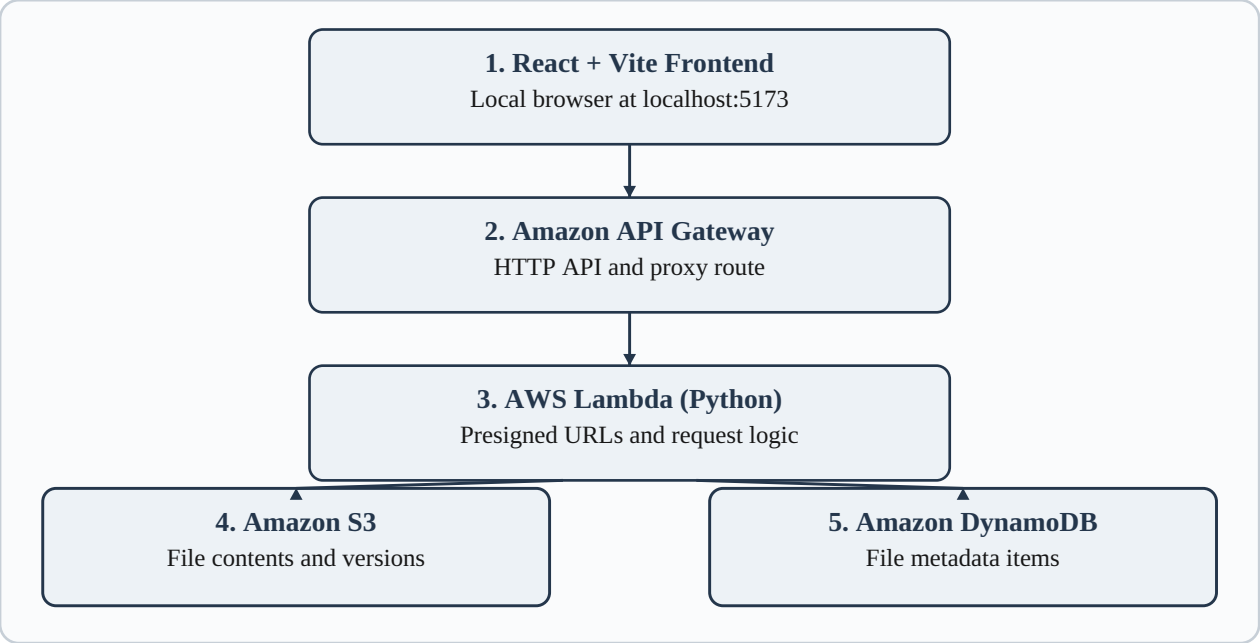


Figure 3.1: Logical architecture of Smart Cloud File Manager.

3.2 Main Features and Functional Capabilities

- **Upload:** The frontend requests an upload URL, sends the file directly to S3, and then confirms the upload so metadata can be stored.
- **File listing:** Lambda scans the metadata table, filters records marked as deleted, and returns the active file entries to the frontend.
- **Download:** The application asks Lambda for a temporary download URL associated with a file ID; the browser then downloads the object from S3.
- **Versioning foundation:** S3 Versioning is enabled. The backend contains routes for listing versions and restoring an earlier version, but these paths still require end-to-end testing.
- **Private storage:** Block Public Access remains enabled. S3 CORS allows the local browser origin for required object operations without making the bucket public.

3.3 Technologies Used and Cloud Infrastructure Stack

Technology / Service	Role in the Project
React and Vite	Browser interface for local development.
Python 3.12 and boto3	Run backend logic and call AWS services through boto3.
Amazon API Gateway HTTP API	Expose HTTP routes and invoke the Lambda function.
AWS Lambda	Execute backend logic on demand.
Amazon S3	Store file objects and preserve versions.
Amazon DynamoDB	Store file ID, name, S3 key, size, time, and status.
AWS IAM	Grant Lambda scoped permissions for AWS resource operations.
npm and Vite environment variables	Manage frontend packages and API URL configuration.

3.4 Serverless Cloud Design: API Gateway, Lambda, S3 and DynamoDB

The HTTP API SmartFileManagerHTTPAPI forwards the ANY /{proxy+} route to SmartFileManagerAPI. The Lambda handler is configured as lambda_function.handler and uses the SmartFileManagerLambdaRole execution role. BUCKET_NAME, TABLE_NAME, and ALLOWED_ORIGIN configure the backend; VITE_API_BASE_URL configures the frontend API endpoint.

3.5 System Workflow and Presigned URL Lifecycle

1. Select a file. The frontend sends the filename, content type, and size to the API.

2. Generate a URL. Lambda validates the request and returns a temporary S3 PUT URL.

3. Upload to S3. The browser sends the file to S3 using the presigned URL.

4. Record metadata. Lambda verifies the object and writes its metadata to DynamoDB.

5. List or download. The API returns file metadata or a temporary download URL.

3.6 Security, Versioning and Cost Awareness

Block Public Access remains enabled. Presigned URLs expire, but anyone who obtains an unexpired URL can use it for the permitted operation, so these URLs must not be shared in reports. The project has no sign-in or per-user access model and is therefore a demonstration prototype. S3 Versioning may retain older versions and delete markers, which can add storage usage. Credits and free allowances are account-specific; two unused unattached EBS volumes were deleted after billing review.

4. RESULTS AND DISCUSSIONS (SCREENSHOTS OF YOUR PROJECT)

The following screenshots document the running application and the uploaded file shown in the interface.

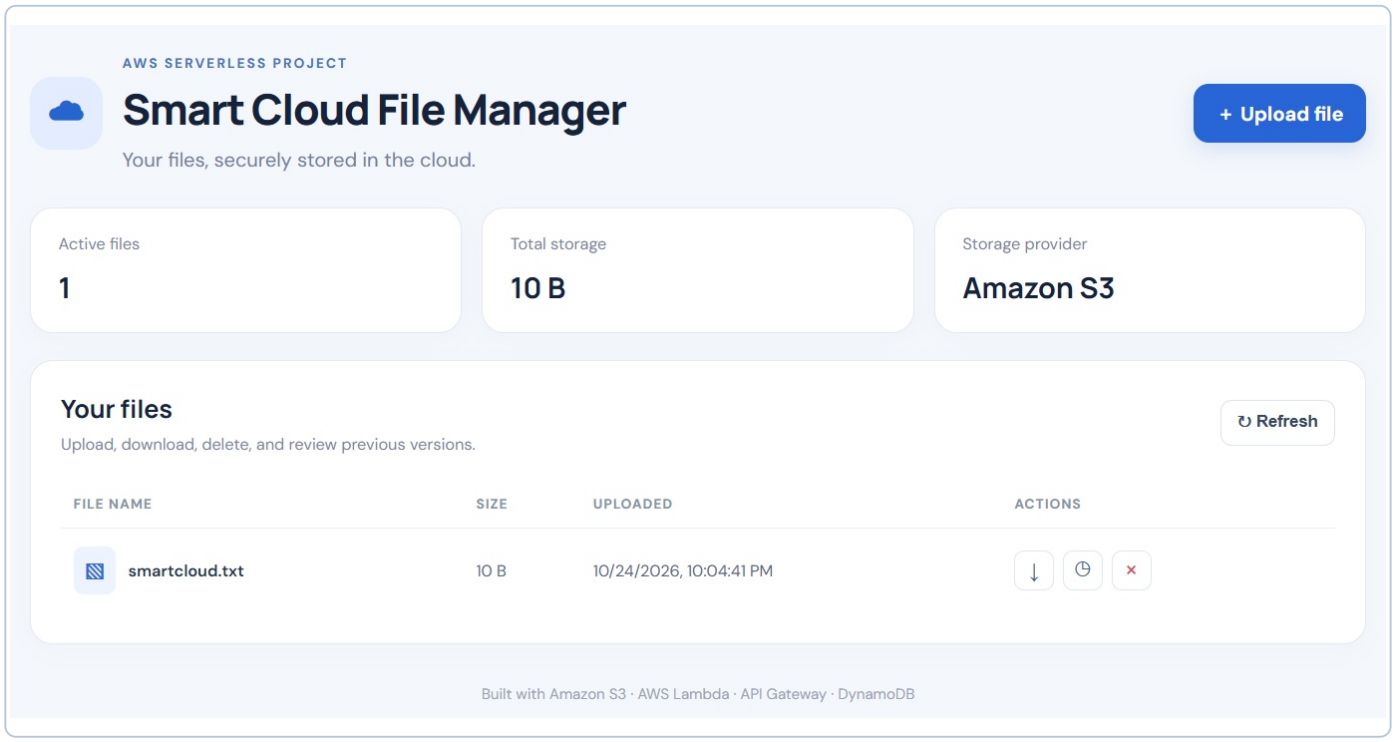


Figure 4.1: Main application interface

Dashboard showing the project title, upload control, storage summary, and file-list area.

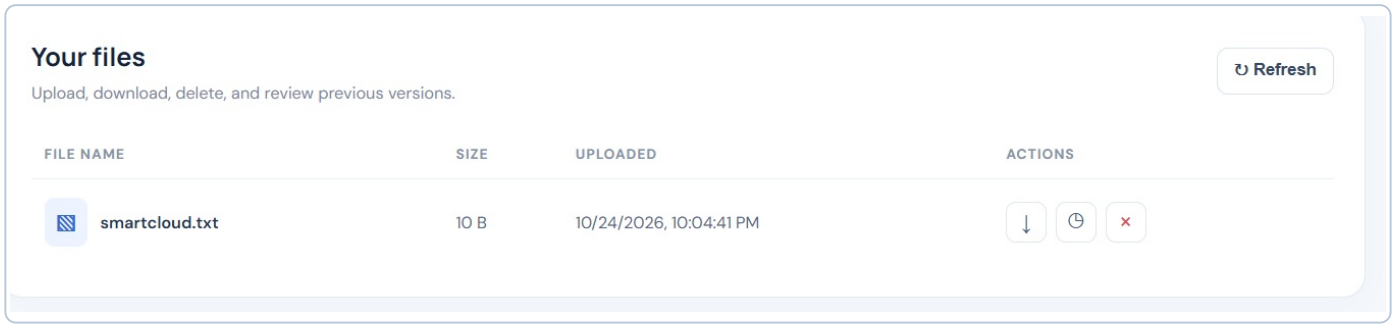


Figure 4.2: File list after upload

The interface lists smartcloud.txt (10 B) and its download, version-history, and delete actions.

4.1 Application Screenshots and Cloud Architecture Verification (continued)

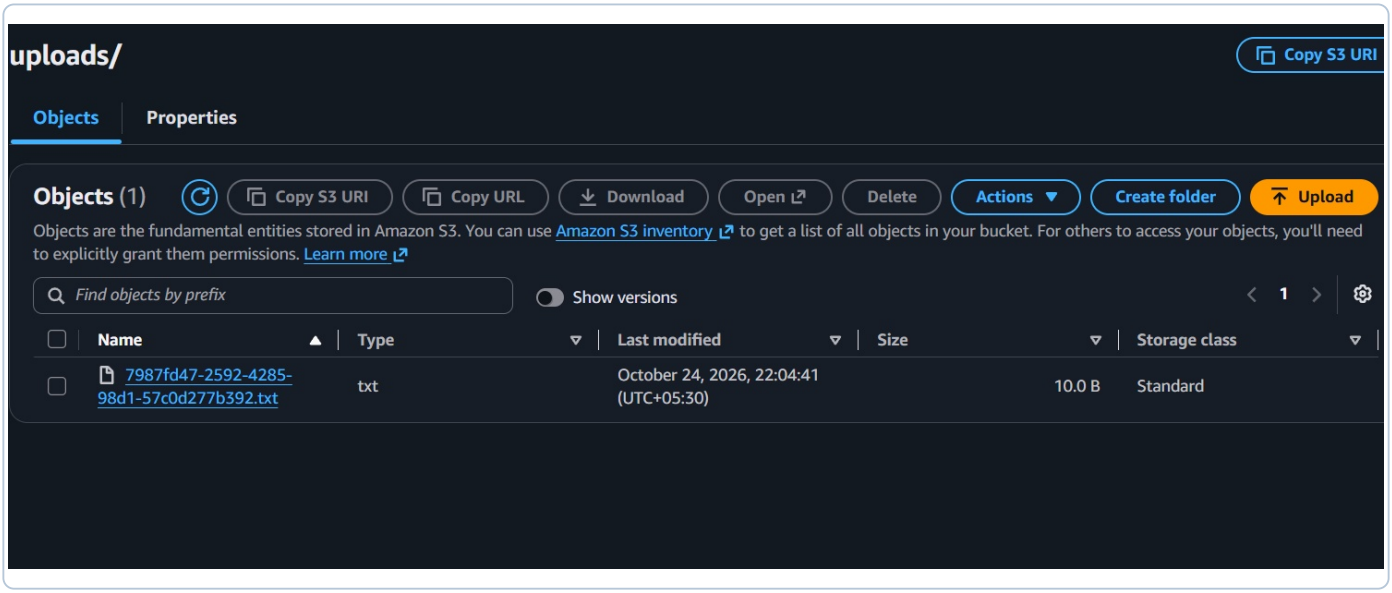


Figure 4.3: Uploaded object in Amazon S3

The uploads/ prefix contains the uploaded text object; its generated object key and 10.0 B size are visible.

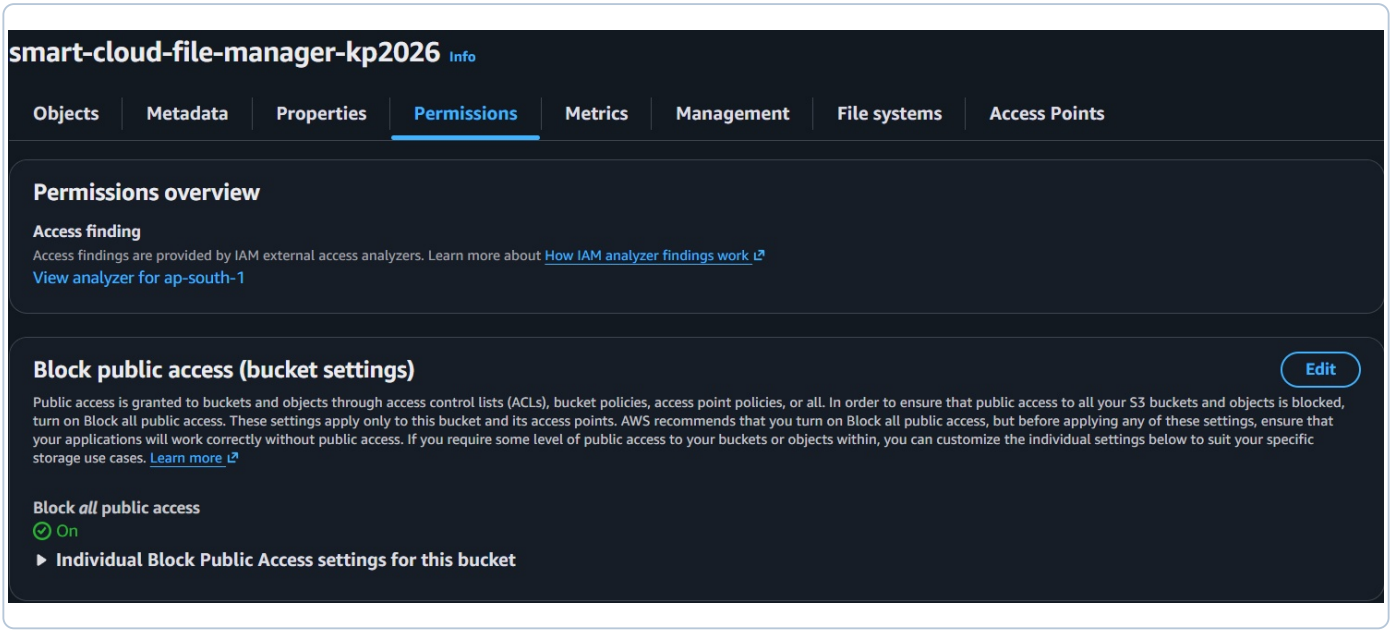


Figure 4.4: S3 permissions and privacy

The bucket permissions page shows Block all public access enabled.

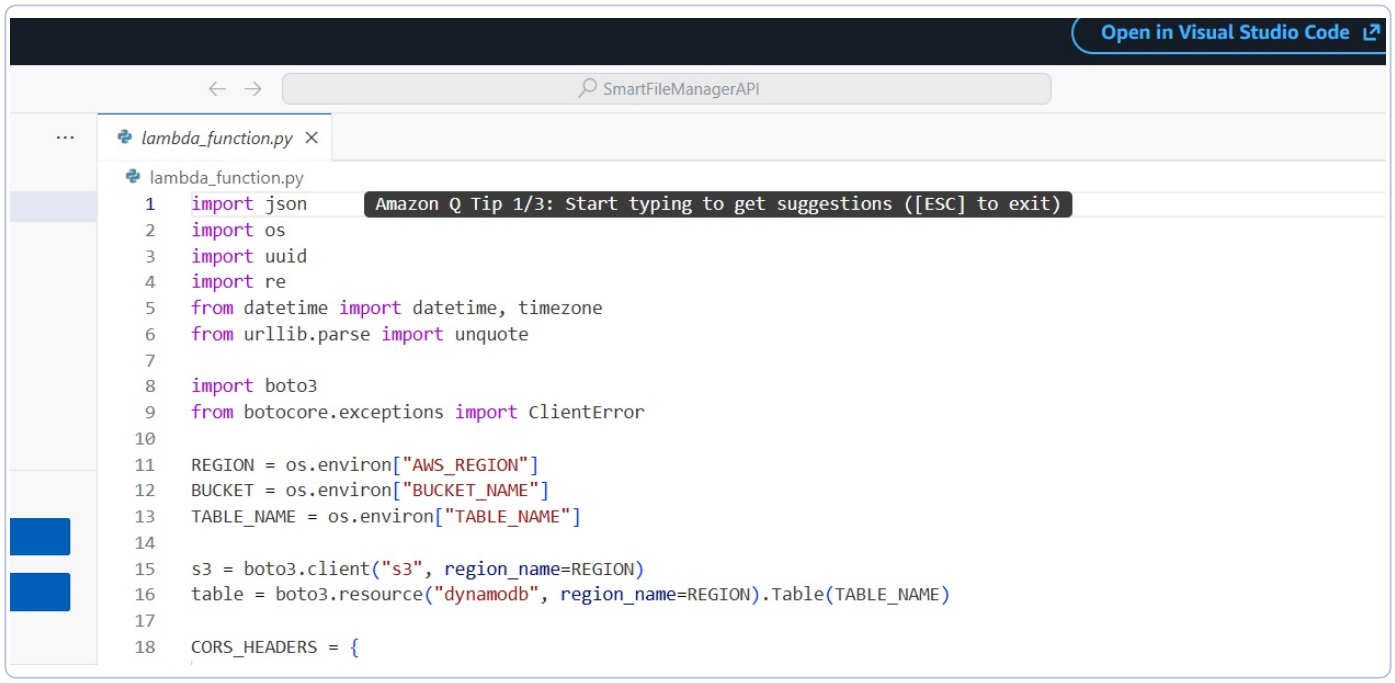


Figure 4.5: Lambda function code source

The SmartFileManagerAPI editor shows the Python source in lambda_function.py; the runtime handler setting is configured separately.



Figure 4.6: Successful Lambda test execution

The console reports Status: Succeeded and a response statusCode of 200. The supplied capture labels the test event as (unsaved) hello-world, rather than TestListFiles.

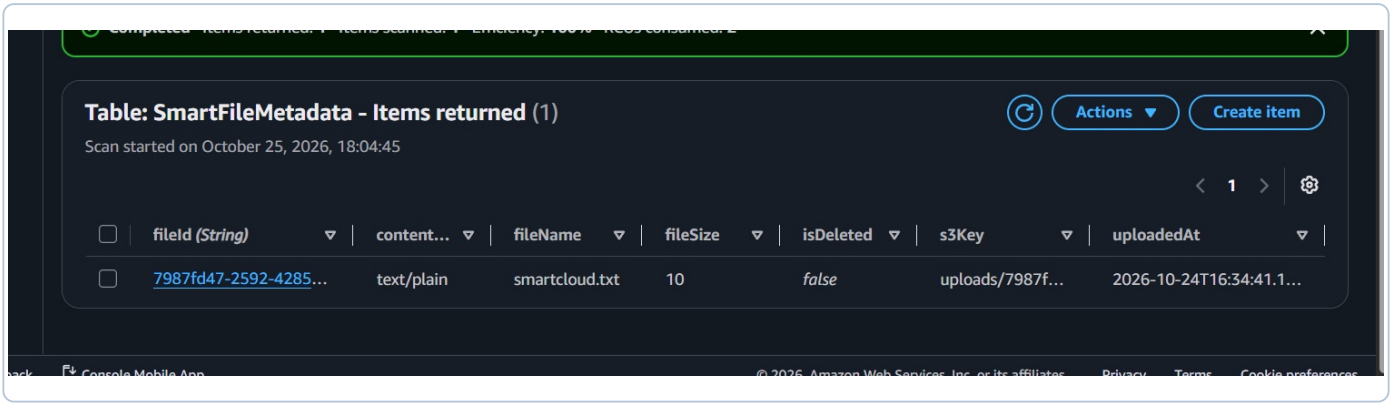


Figure 4.7: DynamoDB metadata table

The SmartFileMetadata scan returned one item. Visible columns include fileId, fileName, fileSize, isDeleted, s3Key, and uploadedAt.

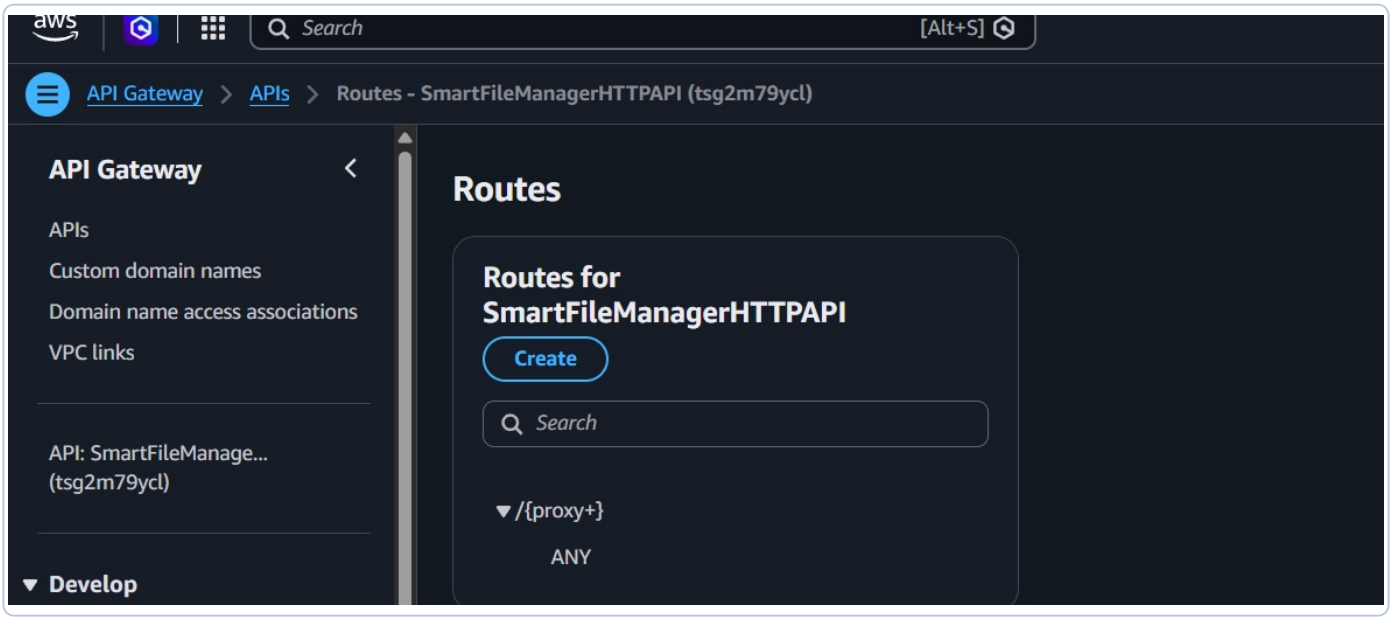


Figure 4.8: API Gateway route

SmartFileManagerHTTPAPI shows the ANY /{proxy+} route on the Routes page.

5. CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

Smart Cloud File Manager demonstrates a serverless approach to file management using React, API Gateway, AWS Lambda, Amazon S3, and DynamoDB. The team created the required AWS resources, connected the frontend to the backend, corrected the Lambda handler configuration, resolved the S3 CORS issue, and verified upload, listing, and download of a small test file. The application provides a practical demonstration of cloud object storage, API integration, serverless request handling, NoSQL metadata, and IAM-based permissions.

The current implementation is a student prototype for Internet Programming. It should not be considered a production-ready file-sharing service because authentication, per-user authorization, comprehensive input validation, operational monitoring, and complete testing of version and delete flows are not yet in place.

5.2 Future Scope

- Add sign-in and per-user file ownership, then enforce authorization for every list, download, restore, and delete operation.
- Add search, sorting, pagination, file-type filters, and upload quotas.
- Complete and test version history, restore, delete, and cleanup behavior when S3 Versioning is enabled.
- Add structured CloudWatch logs, error monitoring, throttling, and alerts for unexpected usage or cost.
- Consider hosting the frontend through an appropriate static site hosting service only after security and cost settings are reviewed.

5.3 Learning Outcomes

- Practised frontend-to-backend communication using an HTTP API and an environment-based API URL.
- Learned how S3 stores objects while DynamoDB stores metadata records.
- Used Lambda tests and CloudWatch logs to diagnose a handler mismatch.
- Learned the role of presigned URLs, CORS, S3 Block Public Access, IAM execution roles, and versioning.
- Reviewed AWS billing and removed two unused, unattached EBS volumes to reduce the chance of future charges.

6. REFERENCES

The following official documentation was used to understand the AWS services and frontend configuration used in the project.

1. Amazon S3 User Guide - Object storage and bucket configuration
https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

2. Amazon S3 - Configure CORS
https://docs.aws.amazon.com/AmazonS3/latest/userguide/ManageCorsUsing.html

3. Amazon S3 - Presigned URLs
https://docs.aws.amazon.com/AmazonS3/latest/userguide/ShareObjectPreSignedURL.html

4. AWS Lambda - Python handler
https://docs.aws.amazon.com/lambda/latest/dg/python-handler.html

5. Boto3 documentation - AWS SDK for Python
https://boto3.amazonaws.com/v1/documentation/api/latest/index.html

6. Amazon API Gateway - HTTP APIs
https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api.html

7. Amazon DynamoDB Developer Guide
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html

8. AWS IAM best practices
https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

9. Vite documentation - Environment variables and modes
https://vite.dev/guide/env-and-mode

10. AWS Billing and Cost Management User Guide
https://docs.aws.amazon.com/cost-management/latest/userguide/what-is-costmanagement.html

Project configuration reference: the SmartFileManagerHTTPAPI HTTP API, SmartFileManagerAPI Lambda function, SmartFileMetadata DynamoDB table, and smart-cloud-file-manager-kp2026 S3 bucket were created by the project team in the ap-south-1 (Mumbai) region. Do not include credentials, session tokens, or complete presigned URLs in the report.

7. ANNEXURE (CODE)

The following listing provides a complete reference implementation of the Lambda route logic shared during project troubleshooting, including the helper routines used by the handler. Before final submission, compare the listing with the final backend/lambda_function.py file on the project computer and update any differences so that the annexure matches the deployed code exactly.

7.1 Backend source listing: backend/lambda_function.py

The runtime handler is configured as lambda_function.handler. The function uses BUCKET_NAME, TABLE_NAME, and ALLOWED_ORIGIN as environment variables.

```
import base64
import json
import os
import re
import uuid
from datetime import datetime, timezone

import boto3
from botocore.exceptions import ClientError

BUCKET = os.environ["BUCKET_NAME"]
TABLE_NAME = os.environ["TABLE_NAME"]
ALLOWED_ORIGIN = os.getenv("ALLOWED_ORIGIN", "http://localhost:5173")

s3 = boto3.client("s3")
table = boto3.resource("dynamodb").Table(TABLE_NAME)


def response(status_code, payload):
    """Return a JSON response with CORS headers for the local frontend."""
    return {
        "statusCode": status_code,
        "headers": {
            "Content-Type": "application/json",
            "Access-Control-Allow-Origin": ALLOWED_ORIGIN,
            "Access-Control-Allow-Methods": "GET,POST,DELETE,OPTIONS",
            "Access-Control-Allow-Headers": "content-type",
        },
        "body": json.dumps(payload, default=str),
    }


def body_json(event):
    """Decode a JSON request body, including base64-encoded API Gateway events."""
    body = event.get("body") or "{}"
    if event.get("isBase64Encoded"):
        body = base64.b64decode(body).decode("utf-8")
    if isinstance(body, dict):
        return body
    return json.loads(body)


def query_params(event):
    return event.get("queryStringParameters") or {}


def clean_filename(value):
    name = os.path.basename(str(value or "file").replace("\\", "/"))
```

7.1 Backend source listing (continued)

```
    name = re.sub(r"[^A-Za-z0-9._ -]", "_", name).strip(" .")
    return (name[:180] or "file")


def handler(event, context):
    method = (
        event.get("requestContext", {}).get("http", {}).get("method")
        or event.get("httpMethod", "GET")
    ).upper()
    path = event.get("rawPath") or event.get("path") or "/"

    if method == "OPTIONS":
        return response(200, {"ok": True})

    try:
        if method == "POST" and path.endswith("/upload-url"):
            data = body_json(event)
            filename = clean_filename(data.get("fileName"))
            content_type = str(data.get("contentType") or "application/octet-stream")[:150]
            size = int(data.get("fileSize") or 0)
            if size < 1 or size > 10 * 1024 * 1024:
                return response(400, {"error": "Choose a file between 1 byte and 10 MB for this demo."})

            file_id = str(uuid.uuid4())
            extension = filename.rsplit(".", 1)[-1] if "." in filename else "bin"
            key = f"uploads/{file_id}.{re.sub(r'[^A-Za-z0-9]', '', extension)[:12]}"
            url = s3.generate_presigned_url(
                "put_object",
                Params={"Bucket": BUCKET, "Key": key, "ContentType": content_type},
                ExpiresIn=300,
            )
            return response(200, {
                "fileId": file_id, "s3Key": key, "fileName": filename,
                "contentType": content_type, "fileSize": size, "uploadUrl": url,
            })

        if method == "POST" and path.endswith("/confirm-upload"):
            data = body_json(event)
            file_id, key = str(data.get("fileId", "")), str(data.get("s3Key", ""))
            if not file_id or not key.startswith("uploads/"):
                return response(400, {"error": "Invalid file details."})
            try:
                obj = s3.head_object(Bucket=BUCKET, Key=key)
            except ClientError:
                return response(400, {"error": "Upload was not found in S3."})
            item = {
                "fileId": file_id,
                "s3Key": key,
```

7.1 Backend source listing (continued)

```
                "fileName": clean_filename(data.get("fileName")),
                "contentType": str(data.get("contentType") or obj.get("ContentType") or "application/
octet-stream"),
                "fileSize": int(obj.get("ContentLength", data.get("fileSize", 0))),
                "uploadedAt": datetime.now(timezone.utc).isoformat(),
                "isDeleted": False,
            }
            table.put_item(Item=item)
            return response(201, {"message": "Upload saved.", "file": item})

        if method == "GET" and path.endswith("/files"):
            result = table.scan()
            items = [i for i in result.get("Items", []) if not i.get("isDeleted", False)]
            while result.get("LastEvaluatedKey"):
                result = table.scan(ExclusiveStartKey=result["LastEvaluatedKey"])
                items.extend([i for i in result.get("Items", []) if not i.get("isDeleted", False)])
            items.sort(key=lambda x: x.get("uploadedAt", ""), reverse=True)
            return response(200, {"files": items})

        if method == "GET" and path.endswith("/download-url"):
            file_id = query_params(event).get("fileId")
            item = table.get_item(Key={"fileId": str(file_id or "")}).get("Item")
            if not item or item.get("isDeleted", False):
                return response(404, {"error": "File not found."})
            url = s3.generate_presigned_url(
                "get_object",
                Params={
                    "Bucket": BUCKET,
                    "Key": item["s3Key"],
                    "ResponseContentDisposition": f'attachment; filename="{item["fileName"]}"',
                },
                ExpiresIn=300,
            )
            return response(200, {"downloadUrl": url})

        if method == "DELETE" and path.endswith("/files"):
            file_id = query_params(event).get("fileId")
            item = table.get_item(Key={"fileId": str(file_id or "")}).get("Item")
            if not item or item.get("isDeleted", False):
                return response(404, {"error": "File not found."})
            # With S3 versioning enabled, delete_object creates a delete marker.
            s3.delete_object(Bucket=BUCKET, Key=item["s3Key"])
            table.update_item(
                Key={"fileId": item["fileId"]},
                UpdateExpression="SET isDeleted = :d",
                ExpressionAttributeValues={":d": True},
            )
            return response(200, {"message": "File moved out of the active list. Its S3 versions may
still exist."})
```

7.1 Backend source listing (continued)

```
        if method == "GET" and path.endswith("/versions"):
            file_id = query_params(event).get("fileId")
            item = table.get_item(Key={"fileId": str(file_id or "")}).get("Item")
            if not item:
                return response(404, {"error": "File not found."})
            result = s3.list_object_versions(Bucket=BUCKET, Prefix=item["s3Key"])
            versions = []
            for v in result.get("Versions", []):
                if v.get("Key") == item["s3Key"]:
                    versions.append({
                        "versionId": v["VersionId"],
                        "lastModified": v["LastModified"].isoformat(),
                        "size": v["Size"],
                        "isLatest": v["IsLatest"],
                    })
            versions.sort(key=lambda v: v["lastModified"], reverse=True)
            return response(200, {"versions": versions})

        if method == "POST" and path.endswith("/restore"):
            data = body_json(event)
            file_id = str(data.get("fileId", ""))
            version_id = str(data.get("versionId", ""))
            item = table.get_item(Key={"fileId": file_id}).get("Item")
            if not item or not version_id:
                return response(404, {"error": "File or version not found."})
            # Copy the selected historical version to the same key as a new current version.
            s3.copy_object(
                Bucket=BUCKET,
                Key=item["s3Key"],
                CopySource={"Bucket": BUCKET, "Key": item["s3Key"], "VersionId": version_id},
            )
            table.update_item(
                Key={"fileId": file_id},
                UpdateExpression="SET isDeleted = :f",
                ExpressionAttributeValues={":f": False},
            )
            return response(200, {"message": "Selected version restored as the current version."})

        return response(404, {"error": "Route not found.", "path": path, "method": method})

    except Exception as exc:
        print("Unhandled error:", repr(exc))
        return response(500, {"error": "Internal server error. Check the Lambda CloudWatch logs."})
```

7.2 Frontend environment configuration

The frontend uses a Vite environment variable to point to the API Gateway Invoke URL. Put this line in frontend/.env and replace the placeholder with the Invoke URL copied from API Gateway. Do not put AWS access keys or secret keys in a frontend environment file.

```
VITE_API_BASE_URL=https://<api-id>.execute-api.<region>.amazonaws.com
```

Lambda environment variables used by the backend

VARIABLE	VALUE / PURPOSE
BUCKET_NAME	smart-cloud-file-manager-kp2026
TABLE_NAME	SmartFileMetadata
ALLOWED_ORIGIN	http://localhost:5173

7.3 Amazon S3 CORS configuration

This configuration permits the local development origin to perform the browser operations used by the prototype. It does not make the bucket public. Keep Block Public Access enabled.

```
[
  {
    "AllowedHeaders": ["*"],
    "AllowedMethods": ["PUT", "GET", "HEAD"],
    "AllowedOrigins": ["http://localhost:5173"],
    "ExposeHeaders": ["ETag"],
    "MaxAgeSeconds": 3000
  }
]
```

--- End of Report ---